

# Profiling & Parallelization

## Lecture 15

Dr. Colin Rundel

# Profiling & Benchmarking

# profvis - lm() demo

```
1 n = 1e6
2 d = tibble(
3   x1 = rt(n, df = 3),
4   x2 = rt(n, df = 3),
5   x3 = rt(n, df = 3),
6   x4 = rt(n, df = 3),
7   x5 = rt(n, df = 3),
8 ) |>
9   mutate(y = -2*x1 - 1*x2 + 0*x3 + 1*x4 + 2*x5 + rnorm(n))
```

```
1 profvis::profvis({
2   lm(y~., data=d)
3 })
```

# profvis - vector demo

```
1 profvis::profvis({  
2   data = data.frame(value = runif(5e4))  
3  
4   data$sum[1] = data$value[1]  
5   for (i in seq(2, nrow(data))) {  
6     data$sum[i] = data$sum[i-1] + data$value[i]  
7   }  
8 })
```

```
1 profvis::profvis({  
2   x = runif(5e4)  
3   sum = x[1]  
4   for (i in seq(2, length(x))) {  
5     sum[i] = sum[i-1] + x[i]  
6   }  
7 })
```

# Benchmarking - bench

```
1 d = tibble(  
2   x = runif(10000),  
3   y = runif(10000)  
4 )  
5  
6 (b = bench::mark(  
7   d[d$x > 0.5, ],  
8   d[which(d$x > 0.5), ],  
9   subset(d, x > 0.5),  
10  filter(d, x > 0.5)  
11 ))
```

# A tibble: 4 × 6

| expression               | min      | median   | `itr/sec` | mem_alloc | `gc/sec` |
|--------------------------|----------|----------|-----------|-----------|----------|
| <bch:expr>               | <bch:tm> | <bch:tm> | <dbl>     | <bch:byt> | <dbl>    |
| 1 d[d\$x > 0.5, ]        | 73µs     | 87.1µs   | 11065.    | 235.86KB  | 31.4     |
| 2 d[which(d\$x > 0.5), ] | 76.1µs   | 95.7µs   | 10174.    | 270.72KB  | 60.2     |
| 3 subset(d, x > 0.5)     | 91.6µs   | 116.4µs  | 8370.     | 288.41KB  | 46.6     |
| 4 filter(d, x > 0.5)     | 239.9µs  | 284.9µs  | 3348.     | 1.48MB    | 18.9     |

# Larger n

```
1 d = tibble(  
2   x = runif(1e6),  
3   y = runif(1e6)  
4 )  
5  
6 (b = bench::mark(  
7   d[d$x > 0.5, ],  
8   d[which(d$x > 0.5), ],  
9   subset(d, x > 0.5),  
10  filter(d, x > 0.5)  
11 ))
```

# A tibble: 4 × 6

| expression               | min      | median   | `itr/sec` | mem_alloc | `gc/sec` |
|--------------------------|----------|----------|-----------|-----------|----------|
| <bch:expr>               | <bch:tm> | <bch:tm> | <dbl>     | <bch:byt> | <dbl>    |
| 1 d[d\$x > 0.5, ]        | 6.76ms   | 7.62ms   | 132.      | 13.3MB    | 107.     |
| 2 d[which(d\$x > 0.5), ] | 8.62ms   | 9.69ms   | 104.      | 24.8MB    | 138.     |
| 3 subset(d, x > 0.5)     | 11.72ms  | 12.74ms  | 79.1      | 24.8MB    | 79.1     |
| 4 filter(d, x > 0.5)     | 9.43ms   | 10.23ms  | 98.4      | 24.8MB    | 63.0     |

# bench - relative results

```
1 summary(b, relative=TRUE)
```

```
# A tibble: 4 × 6
```

|   | expression<br><bch:expr> | min<br><dbl> | median<br><dbl> | `itr/sec`<br><dbl> | mem_alloc<br><dbl> | `gc/sec`<br><dbl> |
|---|--------------------------|--------------|-----------------|--------------------|--------------------|-------------------|
| 1 | d[d\$x > 0.5, ]          | 1            | 1               | 1.67               | 1                  | 1.71              |
| 2 | d[which(d\$x > 0.5), ]   | 1.27         | 1.27            | 1.31               | 1.86               | 2.20              |
| 3 | subset(d, x > 0.5)       | 1.73         | 1.67            | 1                  | 1.86               | 1.26              |
| 4 | filter(d, x > 0.5)       | 1.40         | 1.34            | 1.24               | 1.86               | 1                 |

# Example - BLAS and LAPACK



# Statistics and Linear Algebra

An awful lot of statistics is at its core linear algebra.

For example:

- Linear regression models: find

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

- Principal component analysis
  - Find  $T = XW$  where  $W$  is a matrix whose columns are the eigenvectors of  $X^T X$ .
  - Often solved via SVD - Let  $X = U\Sigma W^T$  then  $T = U\Sigma$ .

# Numerical Linear Algebra

Not unique to Statistics, these are the type of problems that come up across all areas of numerical computing.

- Numerical linear algebra  $\neq$  mathematical linear algebra
- Efficiency and stability of numerical algorithms matter
  - Designing and implementing these algorithms is hard
- Don't reinvent the wheel - common core linear algebra tools (well defined API)

# BLAS and LAPACK

Low level algorithms for common linear algebra operations

## BLAS

- **B**asic **L**inear **A**lgebra **S**ubprograms
- Copying, scaling, multiplying vectors and matrices
- Origins go back to 1979, written in Fortran

## LAPACK

- **L**inear **A**lgebra **P**ackage
- Higher level functionality building on BLAS.
- Linear solvers, eigenvalues, and matrix decompositions
- Origins go back to 1992, mostly Fortran (expanded on LINPACK, EISPACK)

# Modern variants?

Most default BLAS and LAPACK implementations (like R's defaults) are somewhat dated

- Written in Fortran and designed for a single cpu core
- Certain (potentially non-optimal) hard coded defaults (e.g. block size).

Multithreaded alternatives:

- ATLAS - Automatically Tuned Linear Algebra Software
- OpenBLAS - fork of GotoBLAS from TACC at UTexas
- Intel MKL - Math Kernel Library, part of Intel's commercial compiler tools
- cuBLAS / Magma - GPU libraries from Nvidia and UTK respectively
- Accelerate / vecLib - Apple's framework for GPU and multicore computing

# R & BLAS / LAPACK

```
1 sessionInfo()
```

```
R version 4.5.2 (2025-10-31)
```

```
Platform: aarch64-apple-darwin20
```

```
Running under: macOS Tahoe 26.3
```

```
Matrix products: default
```

```
BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.f
```

```
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib
```

```
locale:
```

```
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
```

```
time zone: America/New_York
```

```
tzcode source: internal
```

```
attached base packages:
```

```
[1] parallel stats graphics grDevices utils datasets
```

```
[7] methods base
```

```
other attached packages:
```

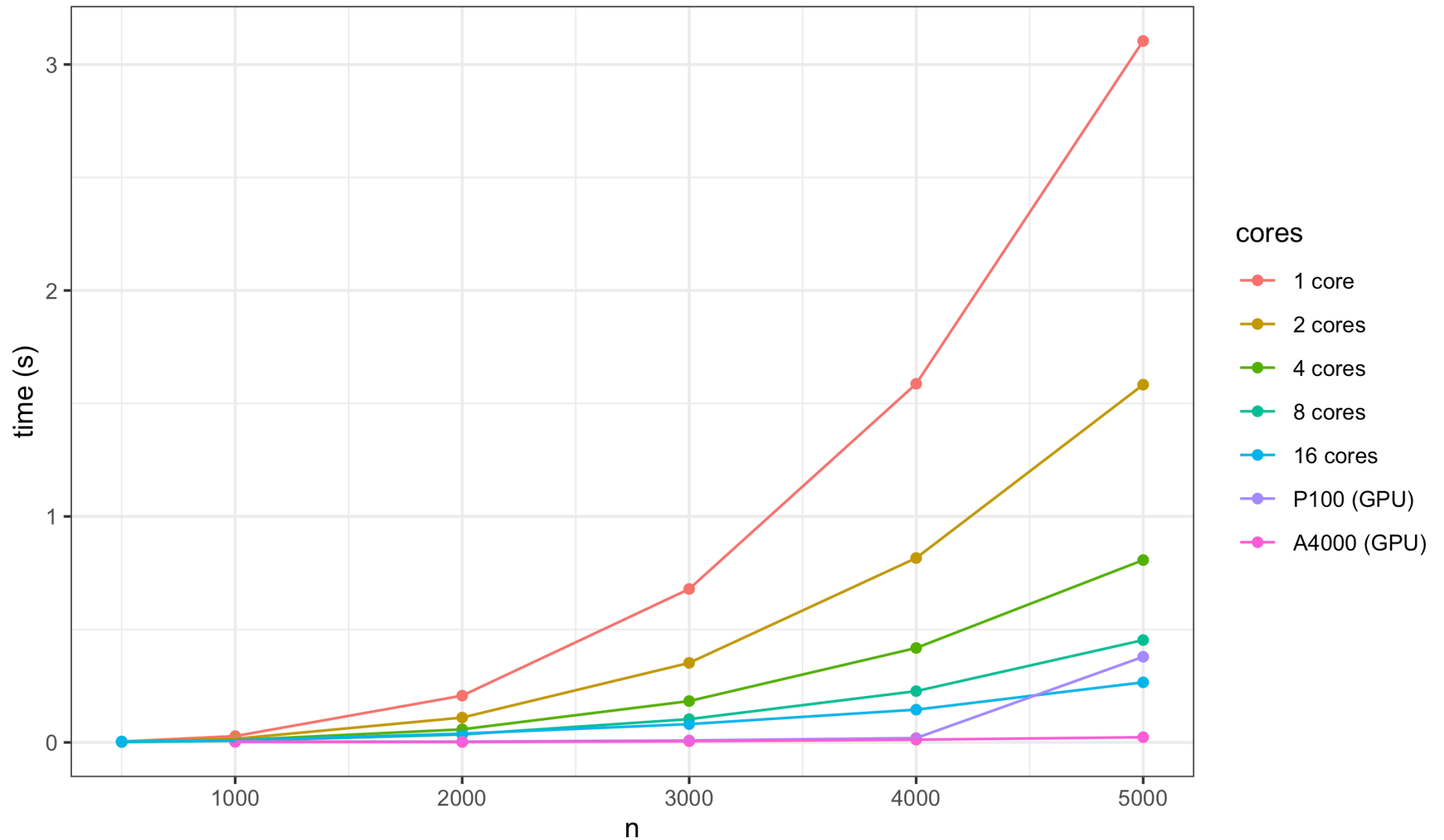
```
[1] lubridate_1.9.5 forcats_1.0.1 stringr_1.6.0 dplyr_1.2.0
```

# OpenBLAS Matrix Multiply Performance

```
1 x = matrix(runif(5000^2),ncol=5000)
2
3 sizes = c(100,500,1000,2000,3000,4000,5000)
4 cores = c(1,2,4,8,16)
5
6 sapply(
7   cores,
8   function(n_cores) {
9     flexiblas::flexiblas_set_num_threads(n_cores)
10    sapply(
11      sizes,
12      function(s) {
13        y = x[1:s,1:s]
14        system.time(y %*% y)[3]
15      }
16    )
17  }
18 )
```

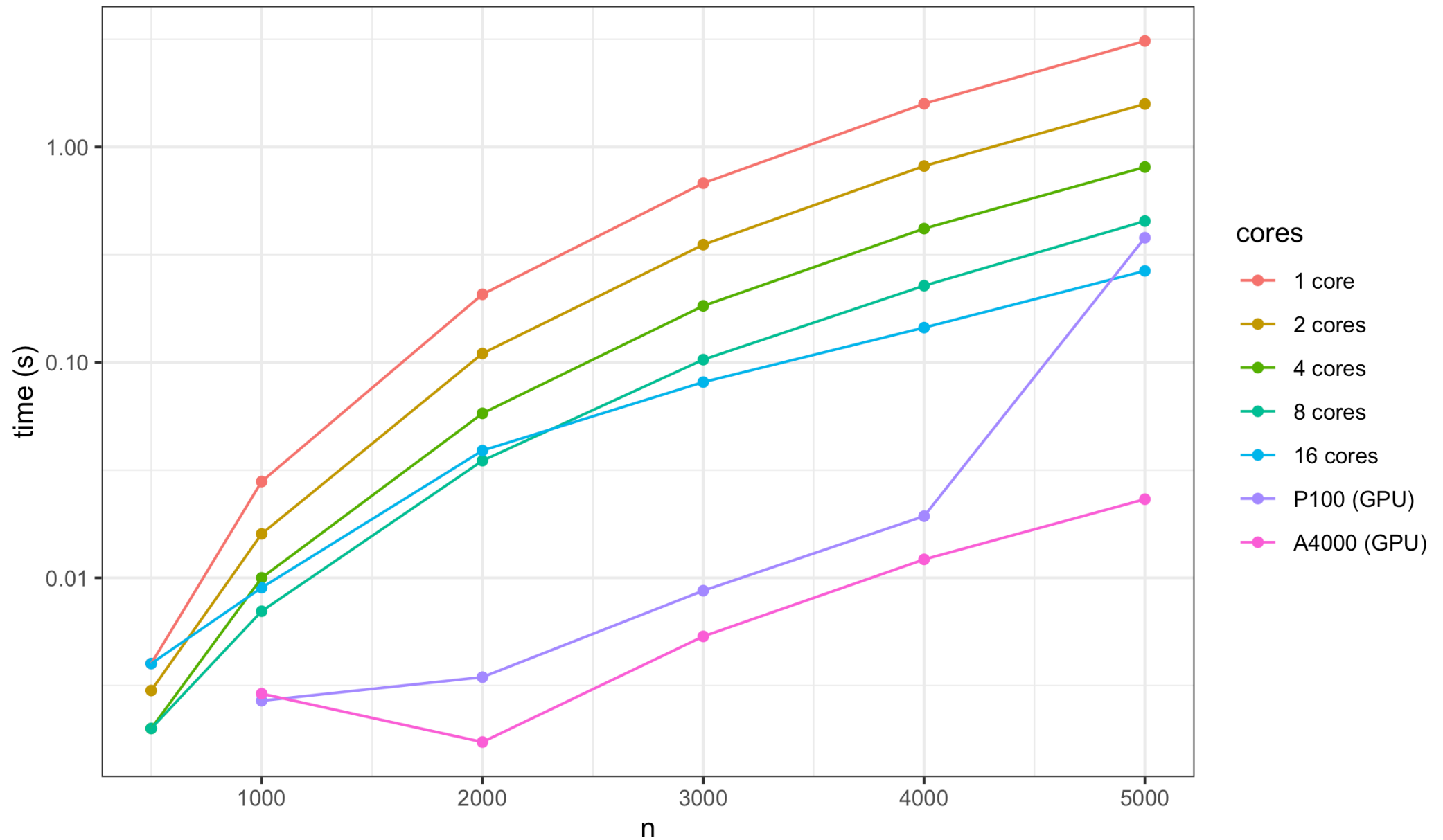
| <b>n</b> | <b>1 core</b> | <b>2 cores</b> | <b>4 cores</b> | <b>8 cores</b> | <b>16 cores</b> |
|----------|---------------|----------------|----------------|----------------|-----------------|
| 100      | 0.000         | 0.000          | 0.000          | 0.000          | 0.000           |
| 500      | 0.004         | 0.003          | 0.002          | 0.002          | 0.004           |
| 1000     | 0.028         | 0.016          | 0.010          | 0.007          | 0.009           |
| 2000     | 0.207         | 0.110          | 0.058          | 0.035          | 0.039           |
| 3000     | 0.679         | 0.352          | 0.183          | 0.103          | 0.081           |
| 4000     | 1.587         | 0.816          | 0.418          | 0.227          | 0.145           |
| 5000     | 3.104         | 1.583          | 0.807          | 0.453          | 0.266           |

## Matrix Multiply of (n x n) matrices





## Matrix Multiply of (n x n) matrices



# Parallelization in R

# parallel

Part of the base packages in R

- tools for the forking of R processes (some functions do not work on Windows)
- Some core functions:
  - `detectCores`
  - `pvec`
  - `mclapply`
  - `mcpipeline` & `mccollect`
- There are many other functions for creating and managing “clusters” of R processes, but we won’t cover those here.

# detectCores

Surprisingly, this detects the number of cores of the current system.

```
1 detectCores()
```

```
[1] 14
```

Take the reported number with a grain of salt; it is important to know something about the hardware you are using. Modern CPUs are complicated and have many features (e.g. hyperthreading, P vs E cores, etc) that can make this number only part of the whole story.

This number also is not aware of other users / processes on the system / job scheduling constraints - so be considerate & conservative when choosing how many cores to use for your parallel computations.

# pvec

Parallelizes a vectorized function call; **FUN** must be a vectorized *pure* function (i.e. no side effects) that returns a vector the same length as **x**.

```
1 system.time(pvec(1:1e7, sqrt, mc.cores = 1))
```

| user  | system | elapsed |
|-------|--------|---------|
| 0.014 | 0.011  | 0.028   |

```
1 system.time(pvec(1:1e7, sqrt, mc.cores = 4))
```

| user  | system | elapsed |
|-------|--------|---------|
| 0.164 | 0.176  | 0.305   |

```
1 system.time(pvec(1:1e7, sqrt, mc.cores = 8))
```

| user  | system | elapsed |
|-------|--------|---------|
| 0.068 | 0.170  | 0.134   |

```
1 system.time(sqrt(1:1e7))
```

| user  | system | elapsed |
|-------|--------|---------|
| 0.013 | 0.014  | 0.026   |

# pvec - bench::system\_time

If you need more precise timing information, `bench::system_time` is a good alternative.

```
1 bench::system_time(Sys.sleep(.5))
```

| process | real  |
|---------|-------|
| 27µs    | 494ms |

```
1 system.time(Sys.sleep(.5))
```

| user  | system | elapsed |
|-------|--------|---------|
| 0.000 | 0.000  | 0.505   |

```
1 bench::system_time(pvec(1:1e7, sqrt, mc.cores = 1))
```

| process | real |
|---------|------|
| 20.3ms  | 20ms |

```
1 bench::system_time(pvec(1:1e7, sqrt, mc.cores = 4))
```

| process | real  |
|---------|-------|
| 166ms   | 183ms |

```
1 bench::system_time(pvec(1:1e7, sqrt, mc.cores = 8))
```

| process | real  |
|---------|-------|
| 229ms   | 261ms |

```
1 bench::system_time(sqrt(1:1e7))
```

| process | real   |
|---------|--------|
| 76.7ms  | 76.6ms |

# Cores by size

```
1 cores = c(1,4,6,8,10)
2 order = 6:8
3 f = function(x,y) {
4   system.time(
5     pvec(1:(10^y), sqrt, mc.cores = x)
6   )[3]
7 }
8
9 res = map(
10   cores,
11   function(x) {
12     map_dbl(order, f, x = x) |>
13     setNames(paste0("10^",order))
14   }
15 ) |>
16 bind_rows() |>
17 mutate(cores = cores) |>
18 relocate(cores)
```

```
1 res
```

```
# A tibble: 5 × 4
  cores `10^6` `10^7` `10^8`
  <dbl>   <dbl>   <dbl>   <dbl>
1     1 0.00200 0.0180 0.257
2     4 0.0300 0.104 1.48
3     6 0.0320 0.0980 0.896
4     8 0.0360 0.0980 1.02
5    10 0.0260 0.103 0.955
```

# mclapply

Implements a parallelized version of `lapply`. It relies on forking and hence is not available on Windows unless `mc.cores = 1`.

```
1 system.time(rnorm(1e7))
```

| user  | system | elapsed |
|-------|--------|---------|
| 0.143 | 0.004  | 0.148   |

```
1 system.time(unlist(mclapply(1:10, function(x) rnorm(1e6), mc.cores = 2)))
```

| user  | system | elapsed |
|-------|--------|---------|
| 0.225 | 0.113  | 0.246   |

```
1 system.time(unlist(mclapply(1:10, function(x) rnorm(1e6), mc.cores = 4)))
```

| user  | system | elapsed |
|-------|--------|---------|
| 0.186 | 0.108  | 0.141   |

```
1 system.time(unlist(mclapply(1:10, function(x) rnorm(1e6), mc.cores = 8)))
```

| user  | system | elapsed |
|-------|--------|---------|
| 0.183 | 0.143  | 0.138   |

```
1 system.time(unlist(mclapply(1:10, function(x) rnorm(1e6), mc.cores = 10)))
```

| user  | system | elapsed |
|-------|--------|---------|
| 0.188 | 0.169  | 0.150   |



# mcparrallel

Asynchronous evaluation of an R expression in a separate process

```
1 m = mcparrallel(rnorm(1e6))  
2 n = mcparrallel(rbeta(1e6,1,1))  
3 o = mcparrallel(rgamma(1e6,1,1))
```

```
1 str(m)
```

List of 2

```
$ pid: int 7718  
$ fd : int [1:2] 5 8  
- attr(*, "class")= chr [1:3] "parallelJob" "childProcess" "process"
```

```
1 str(n)
```

List of 2

```
$ pid: int 7719  
$ fd : int [1:2] 6 10  
- attr(*, "class")= chr [1:3] "parallelJob" "childProcess" "process"
```

# mccollect

Checks `mcparallel` objects for completion - if finished, the values are returned, otherwise the calling R process is blocked until they are done.

```
1 str(mccollect(list(m,n,o)))
```

List of 3

```
$ 7718: num [1:1000000] 0.687 -1.815 0.216 -1.36 1.752 ...  
$ 7719: num [1:1000000] 0.416 0.302 0.992 0.133 0.537 ...  
$ 7720: num [1:1000000] 0.267 0.159 2.867 2.054 1.116 ...
```

# mccollect - waiting

If you don't want to block the calling R process, you can use the `wait = FALSE` argument. This will return `NULL` if the process is not yet complete.

```
1 f = function(n) {  
2   Sys.sleep(5)  
3   mean(rnorm(n))  
4 }  
5 p = mcpipeline(f(1e5))
```

```
1 mccollect(p, wait = FALSE)
```

`NULL`

```
1 mccollect(p, wait = FALSE, timeout = 5)
```

`$`7728``

`[1] -0.001928822`

```
1 mccollect(p, wait = FALSE)
```

Warning in selectChildren(jobs, timeout): cannot wait for child 7728  
as it does not exist

`NULL`

# Aside - RNG and Parallelization

Random number generation in parallel contexts requires special consideration:

- Multiple processes using the same RNG seed produce identical “random” sequences
- Introducing correlation into our RNG invalidates statistical results (e.g., bootstrap, Monte Carlo)
- For most `parallel` package functions, the default behavior ensures proper RNG handling, but be aware of this issue when writing custom parallel code.
- “Reproducibility” can be a function of both your data and core / worker size.
- When using `mirai`, pass `seed = <integer>` to `daemons()` for fully reproducible parallel RNG streams across runs.

# Example

```
1 set.seed(1234)
2 mclapply(
3   1:4, function(x) sd(rnorm(100)), mc.cores = 3
4 ) |>
5 unlist()
```

```
[1] 0.9716484 1.0260644 1.0304322 1.0413186
```

```
1 set.seed(1234)
2 mclapply(
3   1:4, function(x) sd(rnorm(100)), mc.cores = 3
4 ) |>
5 unlist()
```

```
[1] 1.0510987 0.9721524 0.9445127 1.0336663
```

```
1 set.seed(1234)
2 mclapply(
3   1:4, function(x) sd(rnorm(100)),
4   mc.cores = 3, mc.set.seed = FALSE
5 ) |>
6 unlist()
```

```
[1] 1.004405 1.004405 1.004405 1.032187
```

```
1 set.seed(1234)
2 mclapply(
3   1:4, function(x) sd(rnorm(100)),
4   mc.cores = 3, mc.set.seed = FALSE
5 ) |>
6 unlist()
```

```
[1] 1.004405 1.004405 1.004405 1.032187
```

```
1 RNGkind("L'Ecuyer-CMRG")
2 set.seed(1234)
3 mclapply(
4   1:4, function(x) sd(rnorm(100)), mc.cores = 3
5 ) |>
6 unlist()
```

```
[1] 0.9350555 1.1025750 1.0046105 0.9288966
```

```
1 RNGkind("L'Ecuyer-CMRG")
2 set.seed(1234)
3 mclapply(
4   1:4, function(x) sd(rnorm(100)), mc.cores = 3
5 ) |>
6 unlist()
```

```
[1] 0.9350555 1.1025750 1.0046105 0.9288966
```

# parallel alternatives & predecessors

# Historical landscape

## Early packages

- `snow` (2003) - Simple Network of Workstations, cluster-based parallelization
- `multicore` (2009-2014) - Fork-based parallelization for Unix-like systems
- `foreach` (2009) - Iterative parallel execution with pluggable backends

## Consolidation

- `parallel` - Merged `snow` and `multicore` into base R (R 2.14.0)

## Modern alternatives

- `foreach` + backends (`doParallel`, `doMC`, `doFuture`) - Still widely used w/ flexible backends
- `future` (2015) - Modern async framework, support for local/remote/cluster computing
- `mirai` (2022) - Lightweight async evaluation, powers modern tidyverse parallelization

# mirai

is a modern framework for asynchronous parallel computing in R

- Built on nanonext & NNG messaging (IPC, TCP, TLS)
- Scales to millions of tasks with minimal overhead
- Transparent evaluation with error handling
- Works locally, remote (SSH), and on HPC clusters

Core functions:

- `mirai()` - evaluate R expressions asynchronously
- `daemons()` - persistent background processes
- `mirai_map()` - parallel map operations



# Basic `mirai()` usage

Just like `mcpaallel`, we can evaluate R expressions without blocking the session:

```
1 library(mirai)
2 m = mirai(
3   {
4     Sys.sleep(time)
5     rnorm(5L, mean)
6   },
7   time = 2L,
8   mean = 4.5
9 )
```

```
1 m
```

```
< mirai [] >
```

```
1 m$data
```

```
'unresolved' logi NA
```

```
1 unresolved(m)
```

```
[1] TRUE
```

```
1 m[]
```

```
[1] 2.947543 3.294553 5.204873 4.860101 4.
```

```
1 m$data
```

```
[1] 2.947543 3.294553 5.204873 4.860101 4.
```

```
1 m
```

```
< mirai [$data] >
```

# mirai objects

A mirai is either unresolved if the result has yet to be received, or resolved if it has. `unresolved()` is a helper function to check the state of a mirai.

The result of a mirai `m` is available at `m$data` once it has resolved. Normally this will be the return value of the evaluated expression. If the expression errored, crashed, or timed out then this will be an 'errorValue' instead. See the section Error Handling below.

Rather than repeatedly checking `unresolved(m)`, it is more efficient to wait for and collect its value by using `m[]`.

...

If execution in a mirai fails, the error message is returned as a character string of class 'miraiError' and 'errorValue' to facilitate debugging.

`is_mirai_error()` may be used to test for mirai execution errors.

# Error handling

Failed evaluations return `miraiError` objects,

```
1 m = mirai({  
2   stop("Something went wrong!")  
3 })
```

```
1 m[]
```

```
'miraiError' chr Error: Something went wrong!
```

```
1 is_mirai_error(m$data)
```

```
[1] TRUE
```

```
1 m$data$message
```

```
[1] "Something went wrong!"
```

```
1 m$data$stack.trace
```

```
[[1]]  
stop("Something went wrong!")
```

```
1 m$data$condition.class
```

```
[1] "simpleError" "error"      "condition"
```

# Notes on evaluation

The expression passed to `mirai()` will be evaluated in *a separate R process in a clean environment (not the global environment)*. Any variables, functions or other objects that are needed must be supplied via `...` and/or `.args`.

- Use `...` for simple arguments that are substituted into the expression before interpretation
- Use `.args` for functions, large objects, etc.

Similarly, if you are using functions from non-base packages you must use namespaced calls (e.g. `dplyr::select()`), or else the package should be loaded explicitly as part of `.expr`.

Note that when using daemons the function `everywhere()` can be used to evaluate an expression on all connected daemons; this can be useful to load packages or set options.

# daemons ( )

We can create persistent background processes to handle our `mirai()` requests, this reduces startup overhead for each task.

- `daemons(sync = TRUE)` runs all `mirai` calls synchronously in the current process — useful for debugging

```
1 daemons(0)
2 a = mirai({
3   Sys.sleep(3)
4 })
5 b = mirai({
6   Sys.sleep(2)
7 })
8 c = mirai({
9   Sys.sleep(1)
10 })
11
12 system.time(
13   call_mirai(list(a, b, c))
14 )
```

| user  | system | elapsed |
|-------|--------|---------|
| 0.000 | 0.001  | 3.080   |

```
1 daemons(1)
2 a = mirai({
3   Sys.sleep(3)
4 })
5 b = mirai({
6   Sys.sleep(2)
7 })
8 c = mirai({
9   Sys.sleep(1)
10 })
11
12 system.time(
13   call_mirai(list(a, b, c))
14 )
```

| user  | system | elapsed |
|-------|--------|---------|
| 0.000 | 0.000  | 5.972   |

```
1 daemons(3)
2 a = mirai({
3   Sys.sleep(3)
4 })
5 b = mirai({
6   Sys.sleep(2)
7 })
8 c = mirai({
9   Sys.sleep(1)
10 })
11
12 system.time(
13   call_mirai(list(a, b, c))
14 )
```

| user  | system | elapsed |
|-------|--------|---------|
| 0.000 | 0.000  | 2.959   |

# with\_daemons() & local\_daemons()

Scoped daemon management using named compute profiles — a cleaner alternative to manually calling `daemons(0)` to clean up.

`with_daemons()` evaluates an expression within a profile:

```
1 daemons(4, .compute = "work")
2
3 result = with_daemons("work", {
4   mirai_map(1:8, \(x) mean(rnorm(1e6))))[]
5 })
```

`local_daemons()` activates a profile for the current scope:

```
1 f = function() {
2   local_daemons("work")
3   mirai_map(1:4, \(x) mean(rnorm(1e6))))[]
4 }
```

Multiple named profiles (e.g. `"cpu"` and `"gpu"`) can coexist without conflicting

# call\_mirai() vs race\_mirai()

```
1 daemons(4)
```

Call waits for all tasks to complete

```
1 a = mirai({
2   Sys.sleep(3)
3   "A"
4 })
5 b = mirai({
6   Sys.sleep(2)
7   "B"
8 })
9 c = mirai({
10  Sys.sleep(1)
11  "C"
12 })
13
14 call_mirai(list(a, b, c))
15 c(a$data, b$data, c$data)
```

```
[1] "A" "B" "C"
```

Race returns the index of the first completed task

```
1 a = mirai({
2   Sys.sleep(3)
3   "A"
4 })
5 b = mirai({
6   Sys.sleep(2)
7   "B"
8 })
9 c = mirai({
10  Sys.sleep(1)
11  "C"
12 })
13
14 ml = list(a, b, c)
15 (i = race_mirai(ml))
```

```
[1] 3
```

```
1 ml[[i]]$data
```

```
[1] "C"
```

```
1 c(a$data, b$data, c$data)
```

```
[1] NA  NA  "C"
```

# mirai\_map()

This function is similar to `purrr::map()`, but instead of a list of values, returns a 'mirai\_map' object, which is a list of mirai.

A `mirai_map()` call returns almost immediately. The results of a mirai\_map x may be collected using `x[]` or `collect_mirai(x)`, the same as for a mirai. This waits for all asynchronous operations to complete if still in progress.

## Key advantages

1. Returns immediately with all evaluations taking place asynchronously. Printing a 'mirai map' object shows the current completion progress.
2. The 'promise' argument allows a promise action to be registered against each iteration. This can be used to perform side-effects when each iteration completes (such as checkpointing or recording a progress update).
3. Returns evaluation errors as 'miraiError' or 'errorValue' as the case may be, rather than causing the entire map to fail. This allows more efficient recovery from partial failure.
4. Does not rely on a 'chunking' algorithm that attempts to split work into batches according to the number of available daemons, as implemented for instance in the parallel package. Chunking cannot take into account varying or unpredictable compute times over the indices, which mirai scheduling is designed to deal with optimally.



# Scheduling Examples

```
1 daemons(4)
2 waits = c(1, 1, 4, 4, 1, 1, 1, 4)
```

```
1 system.time(
2   mirai_map(waits, Sys.sleep)[]
3 )
```

| user  | system | elapsed |
|-------|--------|---------|
| 0.001 | 0.000  | 6.017   |

```
1 system.time(
2   mclapply(waits, Sys.sleep, mc.cores = 4)
3 )
```

| user  | system | elapsed |
|-------|--------|---------|
| 0.001 | 0.025  | 8.023   |

```
1 daemons(4)
2 n = sample(1:8)
3 f = function(n) mean(rnorm(10^n))
```

```
1 system.time(
2   mirai_map(n, f)[]
3 )
```

| user  | system | elapsed |
|-------|--------|---------|
| 0.000 | 0.001  | 2.632   |

```
1 system.time(
2   mclapply(n, f, mc.cores = 2)
3 )
```

| user  | system | elapsed |
|-------|--------|---------|
| 0.030 | 0.014  | 3.099   |

# purrr's `in_parallel()` (experimental)

Enables parallel computation in purrr map functions using mirai,

```
1 system.time({  
2   x = mclapply(  
3     1:10,  
4     \(x) rnorm(1e6),  
5     mc.cores = 5  
6   ) |>  
7   unlist()  
8 })
```

```
   user  system elapsed  
0.369   0.108   0.228
```

```
1 mean(x)
```

```
[1] 7.349909e-05
```

```
1 sd(x)
```

```
[1] 1.000287
```

```
1 daemons(5)  
2 system.time({  
3   x = map(  
4     1:10,  
5     in_parallel(\(x) rnorm(1e6))  
6   ) |>  
7   unlist()  
8 })
```

```
   user  system elapsed  
0.077   0.130   0.281
```

```
1 mean(x)
```

```
[1] 0.0003887193
```

```
1 sd(x)
```

```
[1] 1.000248
```

# Example - Bootstrapping

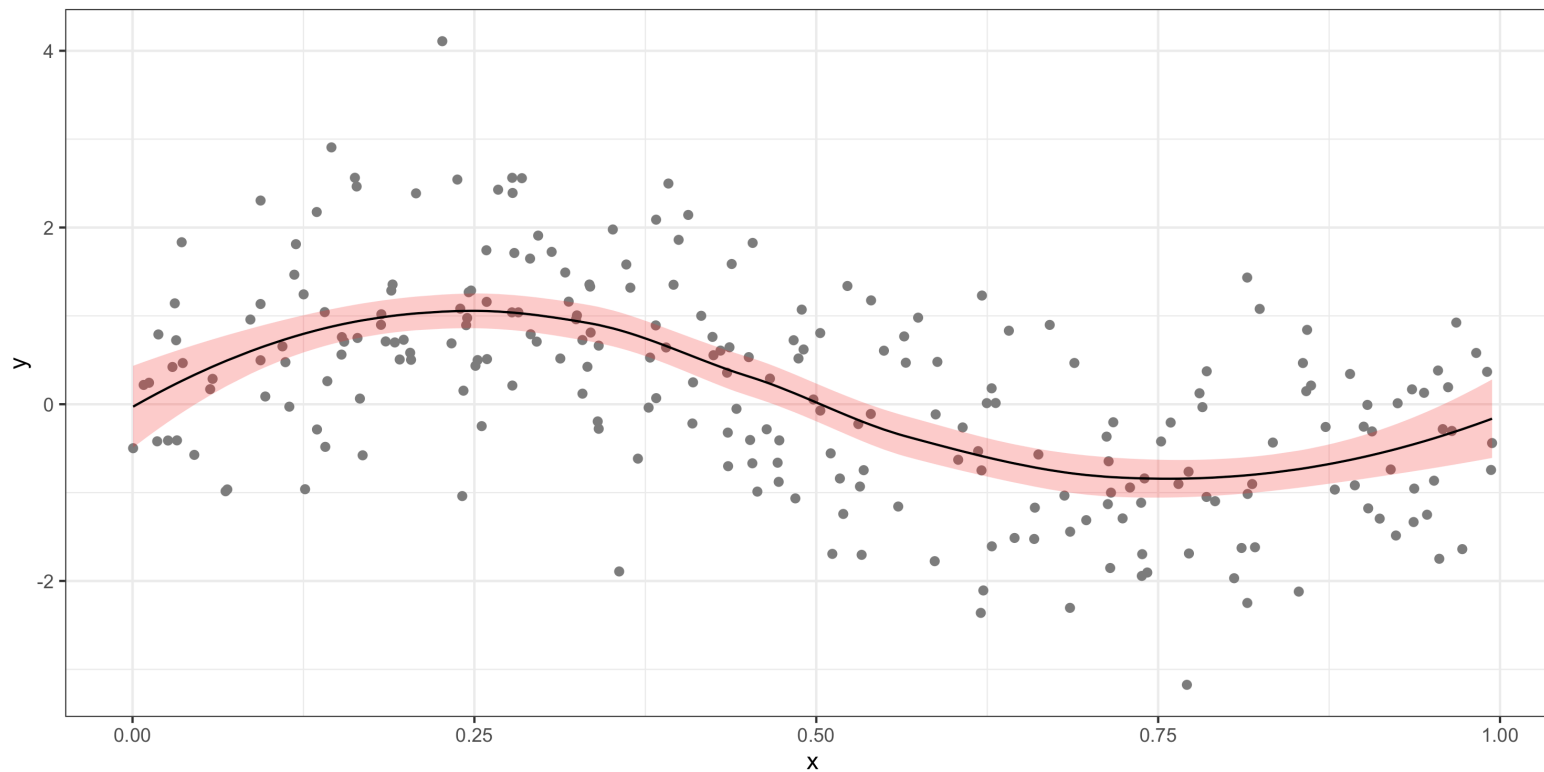
Bootstrapping is a resampling scheme where the original data is repeatedly reconstructed by taking samples of size  $n$  (with replacement) from the original data, and using that to repeat an analysis procedure of interest. Below is an example of fitting a local regression (`loess`) to some synthetic data, we will construct a bootstrap prediction interval for this model.

```
1 set.seed(3212016)
2 d = data.frame(x = runif(250)) |>
3   mutate(y = sin(2*pi*x) + rnorm(length(x)))
4
5 l = loess(y ~ x, data=d)
6 p = predict(l, se=TRUE)
7
8 d = d |> mutate(
9   pred_y = p$fit,
10  pred_y_se = p$se.fit
11 )
```

```

1 ggplot(d, aes(x,y)) +
2   geom_point(color="gray50") +
3   geom_ribbon(
4     aes(ymin = pred_y - 1.96 * pred_y_se,
5         ymax = pred_y + 1.96 * pred_y_se),
6     fill="red", alpha=0.25
7   ) +
8   geom_line(aes(y=pred_y)) +
9   theme_bw()

```



# Bootstrapping Demo

# What to use when?

Optimal use of parallelization / multiple cores is hard, there isn't one best solution

- Don't underestimate the overhead cost
- Experimentation is key
- Measure it or it didn't happen
- Be aware of the trade off between developer time and run time